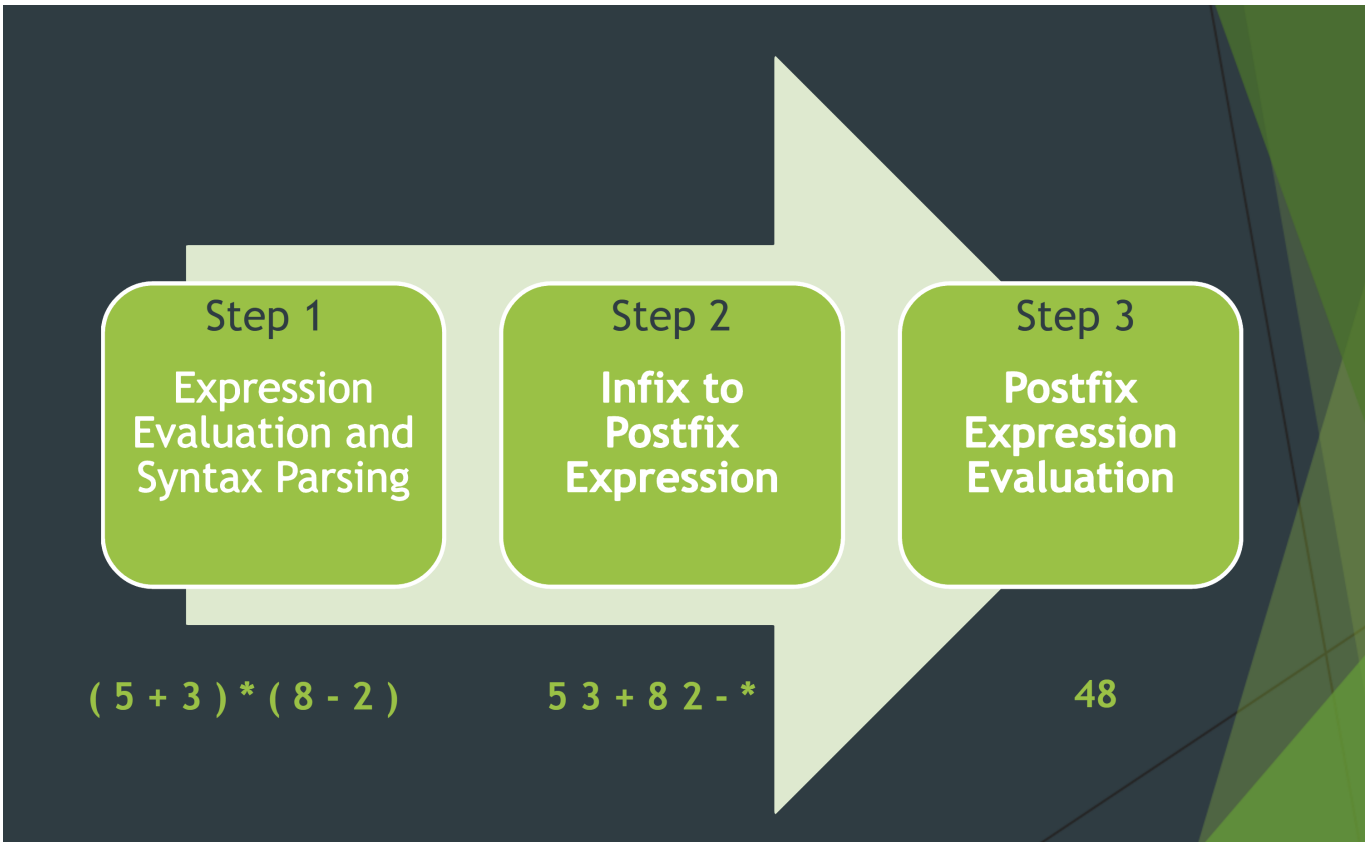
Infix & Postfix

> How do computers parse and read infix expression/equations such as `1+3(2-1)`?

We can use queues and push the `(` & `{` and pop the `)` & `}`, checking if the pairs match everytime we pop. For the order of operations, we use postfix.



- Infix are human readable equations: `1+4`
- Postfix are computer translated equations: `14+`

Process:

- For every complete pair of numbers `1+2`, we put numbers together and the put the operation `+` last. `1+2` is `12+`.

- For nested parathensis, `(5+2)*(6-4)`, we take each number and do the same process. Think of `5+2` as `7`. Then we have a simplified postfix of `7*2` as `72*`. Now, all we need to is to bring back the full equation. `(5+2)*(6-4)` is `52+64-*`.

Another example

`1 + 2 * 3 - 4` is `123*+4-`. Translating `123*+4-`, we do stack operations.

- push `[1, 2, 3]` to stack.

- hit an operator `*`, pop out 2 of the stack

- `2*3 = 6`. push `6` to stack becomes `[1, 6]`

- hit an operator `+`, pop out 2 of the stack
- `1+6 = 7`, push `7` stack becomes `[7]`
- push `4` stack becomes `[7, 4]`
- hit an operator `-`, pop out 2 of the stack
- `7-4 = 3`, push `3` to stack.
- final answer is `[3]`

How to read and check

Think of the compiler reading left-to-right until there are pairs of numbers with an operation.

`52+64-*`

- Read 52+ = 5+2 = 7
- Read 64- = 6-4 = 2
- Translate back 72*
- 72*= 7* 2 = 14.

Example: Translate `1+3(2-1)`

`1+3(2-1)` = `1321-*+`

```
D = A + B * C



Step 1: The Operators in the given Infix Expression : = , + , *

Step 2: The Order of Operators according to their preference : * , + , =

Step 3: Now, convert the first operator * ----- D = A + B C *

Step 4: Convert the next operator + ----- D = A BC* +

Step 5: Convert the next operator = ----- D ABC*+ =

Finally, given Infix Expression is converted into Postfix Expression as follows...

D A B C * + =
```

```
To convert Infix Expression into Postfix Expression using a stack data structure



1. Read all the symbols one by one from left to right in the given Infix
Expression.
```

```
2. If the token is operand, then directly add it to the result (postfix).

3. If the token is '(‘, ‘{‘ then Push it on to the Stack.

4. If the token is ')’, ‘}’

Pop all the contents of stack until respective ‘(‘ or ‘{‘ is popped and add each
poped symbol to the result (postfix).

‘(‘ and ‘{‘ are not added to the result.

5. If the token is operator (+ , - , * , /)

If top of the stack is ‘(‘ or ‘{‘, OR stack is empty, push it.

else, pop the operators which are already on the stack that have higher or equal
precedence than the token, and
add them to the postfix. After then, Push the token.

6. After reading all the symbols, pop all the elements and add them to the
postfix.




A postfix expression can be evaluated using the Stack data structure. To evaluate
a postfix expression
using Stack data structure we can use the following steps...



Read all the symbols one by one from left to right in the given Postfix Expression

If the reading symbol is operand, then push it on to the Stack.

If the reading symbol is operator (+ , - , * , / etc.,), then perform TWO pop
operations and store the
two popped oparands in two different variables (operand1 and operand2). Then
perform reading
symbol operation using operand1 and operand2 and push result back on to the Stack.

Finally! perform a pop operation and display the popped value as final result.
```